

PSD DE SEÑALES ALEATORIAS (GNURADIO)

Daniel Felipe Sarmiento Pilonieta-2202797

Brandon Ferney Caballero Barajas - 2202809

<https://github.com/Danielsarmiento24/CommIIB 1G4>

Abstract

In engineering, the **power spectral density (PSD)** is a key tool used to understand how the energy of a signal is distributed across different frequencies. This communications laboratory introduces the basic concepts of PSD calculation using **GNU Radio**, working with different types of signals such as random bipolar signals, images, and audio.

The practice includes generating patterns that make PSD calculation easier and analyzing how PSD changes under different **samples per symbol (SPS)** settings. It also explores how modifying code blocks can be used to develop applications that involve random signals. By observing signals in both the time and frequency domains, the results show how signal characteristics and block configurations affect the PSD. These experiments provide students with useful insights into how signals behave in real-world communication systems.

Este informe se centra en el estudio de distintos tipos de señales —bipolares, unipolares, de audio e imagen— empleando el software **GNU Radio**. A través de esta plataforma se examinaron sus características espectrales y se compararon las variaciones de la PSD bajo diferentes configuraciones. La posibilidad de interpretar estos resultados y ajustar parámetros como la interpolación y la modulación fue clave para comprender cómo controlar de manera precisa el espectro.

El objetivo principal de esta práctica es **desarrollar la habilidad para comprender, modificar y construir sistemas de análisis de señales** mediante la combinación de bloques de código en GNU Radio, permitiendo diseñar estructuras más avanzadas y eficientes. Este enfoque ofrece una visión integral del análisis y la manipulación de señales en distintos contextos, destacando la relevancia de la PSD en el desarrollo de los sistemas modernos de comunicación.

I. Introducción

El análisis de señales es un pilar fundamental en los campos de las **comunicaciones y el procesamiento de datos**, ya que permite optimizar la transmisión, la recepción y el diseño de sistemas más eficientes. Dentro de estas herramientas, la **Densidad Espectral de Potencia (PSD)** resulta esencial para estudiar cómo se distribuye la energía de una señal en el dominio de la frecuencia.

II. Metodología

En primer lugar, se creó un entorno de trabajo ordenado en GitHub con una nueva rama destinada a la práctica 2. Dentro de esta rama se estructuró el directorio **Practica_2**, que incluye dos subcarpetas principales: **GNURadio**, destinada al desarrollo de los flujogramas, e **Informe**, usada para almacenar y organizar los resultados obtenidos.

Posteriormente, se trabajó con el flujograma **randombinayrectsignal.grc** para

analizar señales binarias aleatorias bipolares con pulsos rectangulares. Se configuraron diferentes valores de muestras por símbolo ($SPS = 1, 4, 8$ y 16), registrando para cada caso la señal en el tiempo, la densidad espectral de potencia (PSD) y los parámetros principales, como la tasa binaria, la frecuencia de muestreo y el ancho de banda.

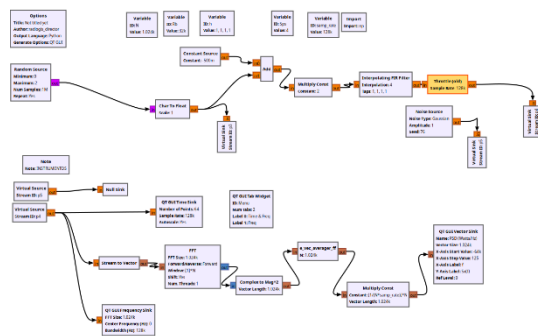


Fig 1. Diagrama de bloques de GNURadio

Luego, se analizó el comportamiento del ruido blanco en tiempo y frecuencia. Para ello, se configuraron las fuentes virtuales p4 y p5, realizando diferentes pruebas y registrando las observaciones correspondientes. Se comprobó si el espectro observado en GNU Radio se asemejaba al espectro plano ideal del ruido blanco.

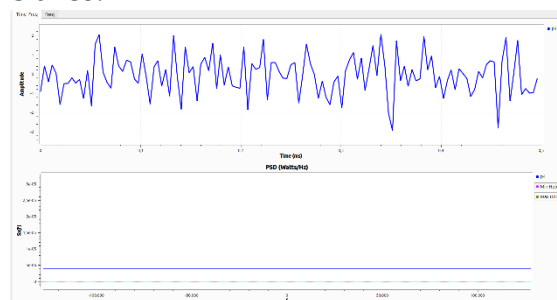


Fig 2. PSD de ruido blanco con $SPS=8$

En la siguiente etapa, se sustituyó la fuente de datos aleatoria por un bloque File Source encargado de leer el archivo rana.jpg. Los bits de la imagen se procesaron con los bloques correspondientes y se analizaron

tanto en el dominio del tiempo como en frecuencia. Esto permitió comparar el espectro de una señal binaria proveniente de un archivo de imagen frente al de una señal generada artificialmente.

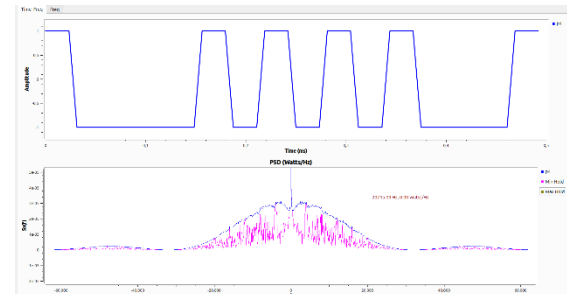


Fig 3. PSD de imagen rana con $SPS=4$

Finalmente, se repitió el procedimiento empleando como fuente el archivo de audio sonido.wav. En este caso, los bits extraídos se procesaron y se compararon con los resultados previos de la señal aleatoria y la imagen. Se observó cómo el contenido de audio genera un espectro diferente, concentrado principalmente en las bajas frecuencias correspondientes a la banda audible.

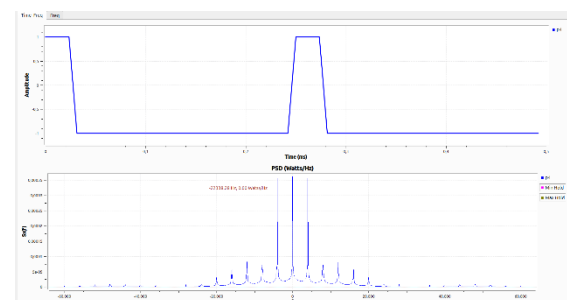


Fig 4. PSD de audio con $SPS=4$

El procedimiento concluyó con la resolución de preguntas de control, donde se identificaron los roles de bloques clave como el filtro interpolador FIR, el bloque de Throttle, y se aplicaron fórmulas relacionadas con el cálculo de ancho de banda, frecuencia de muestreo y resolución espectral. Además, se discutieron las diferencias entre señales binarias bipolares y unipolares, así como las variaciones entre

señales provenientes de ruido, imágenes y audio.

Preguntas de control

a. La combinación de bloques convirtió los bits en una señal continua adecuada para observar la PSD, empaquetando, desempaquetando y convirtiendo los datos a tipo *float*.

b. El *Interpolating FIR Filter* interpoló las muestras según el **SPS**, aumentando la frecuencia de muestreo para mejorar la resolución espectral.

- Si *Interpolation* $\neq$ SPS, cambia la duración de símbolo y se distorsiona la PSD.
- Para analizar *p3* se debía añadir un *QT GUI Sink* ajustando la frecuencia de muestreo.
- El ancho de banda se estimó como $B \approx \frac{Rb}{2}$
- La frecuencia de muestreo en *p3* se calculó como $fs, p3 = fs, p4 / SPS$

c. La PSD del audio mostró más energía en bajas frecuencias por su mayor correlación temporal, a diferencia de la imagen.

d. *Throttle* limitó la velocidad de muestreo para evitar sobrecarga de CPU.

e. Sin conversión a bipolar (0/1) aparece un pico de continua en 0 Hz.

f. El ruido blanco teórico es de ancho de banda infinito, pero en GNU Radio se limitó por la frecuencia de muestreo.

g. La señal binaria también es teóricamente infinita en ancho de banda, pero se observó recortada por el filtrado e interpolación.

h. El número de lóbulos se estimó con $Nl \approx fs/B$, contando doble el lóbulo central.

i. El rango total de frecuencias fue $F_{tot} = \pm(SPS \cdot Rb)/2$

j. La resolución espectral fue $\Delta f = fs/N$

k. Con *Unpack K Bits* en $K=16$, la tasa de bits y la frecuencia de muestreo de salida se multiplicaron por 16.

l. La frecuencia de entrada al *Unpack* se relacionó con el número de lóbulos: $fs, in \approx Nl \cdot B$

m. La frecuencia de salida fue $fs, out = fs, in \times K$

n. La salida de *Char to Float* mantuvo la misma frecuencia de muestreo que la entrada.

o. Con $SPS = 1$ la PSD de la señal binaria se asemejó más a la del ruido blanco.

p. Para forma de diente de sierra se ajustó el parámetro *h* para generar pendiente en cada símbolo.

q. Para codificación Unipolar RZ se modificó *h* y el filtro para incluir retorno a cero a mitad de bit.

r. Para Manchester NRZ se diseñó el filtro con transición a mitad de bit.

s. Para OOK se ajustó *h* y el FIR para que la amplitud dependiera de cada bit (presencia/ausencia).

t. Para BPSK se programó el FIR para producir cambios de fase $\pm\pi$.

u. Para ASK se moduló la amplitud en dos niveles distintos.

v. Para forma de latido se diseñó un filtro que generara picos anchos, ajustando *h*.

w. Para la señal rizada se configuró *h* y el FIR para producir los lóbulos mostrados en la guía.

x. La PSD bipolar no presenta componente DC, mientras que la unipolar sí muestra un pico en 0 Hz.

III. Código de GNURadio

```
"import numpy as np
from gnuradio import gr
```

```
class blk(gr.sync_block):
    def __init__(self, N=1024):
        gr.sync_block.__init__(
            self,
            name='e_vec_averager_ff',
            in_sig=[(np.float32,N)],
            out_sig=[(np.float32,N)])
        self.N = N
        self.rows_counter=0
        self.sum=0

    def work(self, input_items, output_items):
        x=input_items[0]
        y=output_items[0]
        self.sum += np.sum(x, axis=0)
        self.rows_counter += len(x)
        y[:]=self.sum/self.rows_counter
        return len(output_items[0])"
```

El bloque embebido recibe vectores de tamaño fijo N y calcula el promedio acumulado de todos los vectores procesados hasta el momento. Internamente mantiene una suma total y un contador de filas para actualizar el promedio en cada ejecución. De esta manera, suaviza las variaciones de la señal y es útil en aplicaciones como el cálculo de la densidad espectral de potencia. Sin embargo, este bloque no realiza filtrado selectivo ni análisis de la señal, ya que simplemente promedia los vectores de entrada sin distinguir entre diferentes frecuencias o componentes específicas.

IV. Análisis de resultados

Fig 1:

En la parte superior de la figura se aprecia la señal de ruido blanco en el dominio del tiempo.

La forma aleatoria y sin patrones repetitivos confirma la naturaleza impredecible de este tipo de señal. En la parte inferior se observa su densidad espectral de potencia (PSD), la cual se mantiene prácticamente plana a lo largo de todo el rango de frecuencias representado. Este comportamiento es consistente con la teoría, ya que el ruido blanco posee energía distribuida de manera uniforme en el espectro.

El uso de un valor de $SPS = 8$ permitió una representación más detallada de la señal, evitando aliasing y facilitando la verificación del espectro idealmente plano.

Fig 3:

En la parte superior de la figura se observa la señal binaria obtenida a partir de los bits de la imagen. A diferencia del ruido blanco, la forma temporal presenta niveles definidos y transiciones abruptas entre estados, propias de una secuencia de datos estructurados.

En la parte inferior se muestra la densidad espectral de potencia (PSD), la cual exhibe picos de energía más concentrados alrededor de la frecuencia central y una disminución gradual hacia los extremos. Este comportamiento refleja la existencia de correlación en los datos de la imagen, que limita el ancho de banda efectivo en comparación con una señal completamente aleatoria.

El valor de $SPS = 4$ permitió una representación adecuada del espectro, aunque con una resolución menor que la obtenida en el caso del ruido blanco con $SPS = 8$.

Fig 4.

En la parte superior de la figura se observa la señal binaria generada a partir de los bits extraídos del archivo de audio. La forma temporal presenta segmentos de amplitud constante que corresponden a la codificación digital del contenido sonoro. En la parte inferior se muestra la densidad espectral de potencia (PSD), donde la energía se concentra principalmente en las bajas frecuencias, con picos definidos que representan los componentes dominantes del archivo de audio. Este patrón es coherente con las características de la banda audible, en la que la mayor parte de la información útil se encuentra en frecuencias bajas y medias. El uso de $SPS = 4$ permitió capturar las variaciones del espectro sin introducir aliasing, aunque con una resolución menor que la lograda con SPS superiores.

V. Conclusión

El aumento del valor de SPS permite representar con mayor precisión una señal binaria aleatoria bipolar en los dominios del tiempo y la frecuencia. Esto se debe a que una mayor tasa de muestreo mejora la resolución de la señal. Con un SPS bajo ($SPS=1$), la PSD presenta un comportamiento similar al de un ruido blanco, mientras que al incrementar el SPS, las componentes de frecuencia se vuelven más definidas, favoreciendo un análisis más detallado sin afectar la tasa de bits ni la duración de cada bit.

En las señales de audio, la interpolación precisa y la configuración adecuada del número de bits son fundamentales para optimizar la resolución espectral y resaltar picos importantes. Un ajuste correcto en la interpolación ayuda a suavizar las transiciones, asegurando una mejor calidad en el análisis del espectro de la señal.

REFERENCIAS

- [1] O. J. Tijero Rojas, *Laboratorio 3 – PSD de Señales Aleatorias (GNU Radio)*, Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones, Universidad Industrial de Santander, Bucaramanga, Colombia, 2025.
- [2] GNU Radio Project, “Creating Your First Block,” *GNU Radio Wiki*. [En línea]. Disponible en: https://wiki.gnuradio.org/index.php/Creating_Your_First_Block. [Accedido: septiembre 2025].
- [3] GNU Radio Project, “GNU Radio Documentation,” *GNU Radio Official Docs*. [En línea]. Disponible en: https://wiki.gnuradio.org/index.php/Main_Page. [Accedido: septiembre 2025].
- [4] A. V. Oppenheim and A. S. Willsky, *Signals and Systems*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1997.
- [5] S. Haykin, *Communication Systems*, 5th ed. New York, NY, USA: Wiley, 2014.
- [6] GitHub Docs, “Creating and Deleting Branches,” *GitHub Documentation*. [En línea]. Disponible en: <https://docs.github.com/es>. [Accedido: septiembre de 2025].